



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Integrantes:

- José Proaño
- Cristian Robalino
- Darwin Panchez

Tema: Patrón MVC

Carrera: Software

NRC: 23305

Fecha: 10/06/2025

Link:

<https://onlinegdb.com/dt7Nbq3E8>

REQUISITOS:

Para cada RF realizar la revisión de código y explicar a través de la ejecución el funcionamiento de MVC

1. La clase main, llama al View, y al controlador

Main.java (Código):

```
public class Main {  
  
    public static void main(String[] args) {  
        StudentView view = new StudentView();  
        StudentController controller = new StudentController(view);  
        controller.start();  
    }  
}
```

Main.java (Ejecución):

Llama a controller.start() pero no produce salida directamente.

Main.java (Explicación):

¿Qué hace?

- Es el punto de inicio del programa.
- Crea una instancia de la Vista (StudentView).



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

- Crea una instancia del Controlador (StudentController), que recibe la vista.
 - Llama a controller.start() para iniciar la lógica principal.
2. Se hizo el modelo en base al dado, únicamente se adaptando, añadiendo tres nuevos Constructores

Student.java (Código Sin Corregir):

```
public class Student {  
  
    private String rollNo;  
    private String name;  
  
    public Student() {  
        this("", "");  
    }  
  
    public Student(String name, String rollNo) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
  
    public Student(Student student) {  
        this.rollNo = student.getRollNo();  
        this.name = student.getRollNo();  
    }  
  
    public String getRollNo() {  
        return rollNo;  
    }  
    public void setRollNo(String rollNo) {  
        this.rollNo = rollNo;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```



Student.java (Código Corregido):

```
public class Student {  
  
    private String rollNo;  
    private String name;  
  
    public Student() {  
        this("", "");  
    }  
  
    public Student(String name, String rollNo) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
  
    public Student(Student student) {  
        this.rollNo = student.getRollNo();  
        this.name = student.getName();  
    }  
  
    public String getRollNo() {  
        return rollNo;  
    }  
    public void setRollNo(String rollNo) {  
        this.rollNo = rollNo;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Student.java (Ejecución):

No genera salida directa, solo representa el modelo.

Student.java (Explicación):

¿Qué hace?

Representa el modelo de datos, es decir, un estudiante. Contiene:



- Atributos: name y rollNo (número de matrícula).
 - Constructores:
 - Sin parámetros (crea un estudiante vacío).
 - Con nombre y número de matrícula.
 - Constructor de copia de estudiante (Aquí se debe mejorar la línea `this.name = student.getRollNo();` por `this.name = student.getName();`; de lo contrario, al clonar un Student, el campo name recibirá por error el valor de rollNo, haciendo que el nombre del alumno sea idéntico a su número de matrícula, lo cual no corresponde al propósito de un constructor de copia).
 - Métodos get y set para acceder y modificar los atributos.
3. Para el view se creó un método de inserción, el cual simula cómo sería la inserción tradicional

StudentView.java (Código Sin Corregir):



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

```
import java.util.Scanner;

public class StudentView {

    public void printStudentDetails(Student student) {
        System.out.println("Student: ");
        System.out.println("Name: " + student.getName());
        System.out.println("Roll No: " + student.getRollNo());
        System.out.println("");
    }

    public Student inputStudent() {
        String name = "David";
        String rollNo = "1";
        System.out.println("***Create: ");
        System.out.println("Student: ");
        System.out.println("Name: ");
        System.out.println("Input: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Input: " + rollNo);
        System.out.println("***End: ");
        System.out.println("");

        return new Student(name, rollNo);
    }
}
```

StudentView.java (Código Corregido):

```
import java.util.Scanner;

public class StudentView {

    public void printStudentDetails(Student student) {
        System.out.println("Student: ");
        System.out.println("Name: " + student.getName());
        System.out.println("Roll No: " + student.getRollNo());
        System.out.println("");
    }

    public Student inputStudent() {
        String name = "David";
        String rollNo = "1";
        System.out.println("***Create: ");
        System.out.println("Student: ");
        System.out.println("Name: " + name);
        System.out.println("Input: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Input: " + rollNo);
        System.out.println("***End: ");
        System.out.println("");

        return new Student(name, rollNo);
    }
}
```



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

StudentView.java (Ejecución)

printStudentDetails(...)

```
*****Fetching Data*****
Student:
Name: Cristian
Roll No: 10

Student:
Name: Darwin
Roll No: 11

Student:
Name: Jose
Roll No: 12
```

inputStudent()

```
*****Creating Data*****
***Create:
Student:
Name:
Input: Pepe
Roll No: 1
Input: 1
***End:
```

StudentView.java (Explicación):

¿Qué hace?

- Es la interfaz de usuario, aunque muy básica.
- Métodos:
 - printStudentDetails(...): Imprime en consola los datos de un estudiante.
 - inputStudent(): Simula pedir datos al usuario (hardcodea "David", "1") y los imprime.



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

4. El controlador se cambió en su mayoría, haciendo uso únicamente de los métodos que se requieren para hacer un intermediario entre el modelo, y la vista

StudentController.java (Código):

```
import java.util.List;

public class StudentController {

    private StudentDatabase database;
    private StudentView view;

    public StudentController(StudentView view) {
        this.view = view;
        this.database = StudentDatabase.getInstance();
    }

    public void start() {
        System.out.println("*****Fetching Data*****");
        this.fetchStudents();
        System.out.println("*****Creating Data*****");
        this.createStudent();
        System.out.println("*****Updating Data*****");
        this.updateStudent();
    }

    public void fetchStudents() {
        this.updateView();
    }

    public void createStudent() {
        Student student = this.view.inputStudent();
        this.database.postStudent(student);
        this.updateView();
    }

    public void updateStudent() {
        Student oldStudent = this.database.getStudents().get(0);
        Student studentUpdate = new Student(oldStudent);
        studentUpdate.setName("Jon");

        this.database.putStudent(oldStudent, studentUpdate);
        this.updateView();
    }

    private void updateView() {
        List<Student> data = this.database.getStudents();
        for(Student student: data) {
            this.view.printStudentDetails(student);
        }
    }
}
```

StudentController.java (Ejecución):

```
*****Fetching Data*****
```

```
*****Creating Data*****
```

```
*****Updating Data*****
Student:
Name: Jon
Roll No: 10

Student:
Name: Darwin
Roll No: 11

Student:
Name: Jose
Roll No: 12

Student:
Name: Keyko
Roll No: 10

Student:
Name: Alice
Roll No: 11

Student:
Name: Pepe
Roll No: 1
```

StudentController.java (Explicación):

¿Qué hace?

Es el controlador, encargado de la lógica de la aplicación.

- start():
 - Llama a fetchStudents() → muestra los estudiantes iniciales.
 - Llama a createStudent() → agrega un nuevo estudiante (David, 1).
 - Llama a updateStudent() → actualiza el primer estudiante de la lista.
- fetchStudents(): Muestra todos los estudiantes.
- createStudent():



- Crea un estudiante con `view.inputStudent()`.
 - Lo agrega a la base de datos.
 - Actualiza la vista.
 - `updateStudent()`:
 - Toma el primer estudiante de la base de datos.
 - Crea una copia del estudiante.
 - Cambia el nombre a "Jon".
 - Lo reemplaza en la lista (pone al nuevo estudiante en la primera posición).
 - `updateView()`: Muestra todos los estudiantes actuales.
5. Se creó una clase que simula una base de datos, la cual brinda apoyo en la administración de los datos quemados.

StudentDatabase.java (Código):



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

```
import java.util.List;
import java.util.ArrayList;

public class StudentDatabase {

    private static StudentDatabase instance;
    private List<Student> students;

    private StudentDatabase() {
        this.students = new ArrayList<>();
        this.students.add(new Student("Cristian", "10"));
        this.students.add(new Student("Darwin", "11"));
        this.students.add(new Student("Jose", "12"));
        this.students.add(new Student("Keyko", "10"));
        this.students.add(new Student("Alice", "11"));
    }

    public static StudentDatabase getInstance() {
        if(instance == null) {
            instance = new StudentDatabase();
        }
        return instance;
    }

    public List<Student> getStudents() {
        return this.students;
    }

    public void postStudent(Student student) {
        this.students.add(student);
    }

    public void putStudent(Student oldStudent, Student updatedStudent) {
        int index = this.students.indexOf(oldStudent);
        this.students.remove(index);
        this.students.add(0, updatedStudent);
    }

    public void deleteStudent(Student student) {
        this.students.remove(student);
    }
}
```

StudentDatabase.java (Ejecución):

No produce salida directamente, solo gestiona los datos internos más se puede comprobar que los datos se encuentran cargados al momento de presentar el listado de estudiantes.



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

```
Student:
Name: Cristian
Roll No: 10

Student:
Name: Darwin
Roll No: 11

Student:
Name: Jose
Roll No: 12

Student:
Name: Keyko
Roll No: 10

Student:
Name: Alice
Roll No: 11
```

StudentDatabase.java (Explicación):

¿Qué hace?

Simula una base de datos en memoria (singleton):

- getInstance(): asegura que solo haya una instancia (singleton).
- Al inicializar, carga una lista de estudiantes ficticios (Robert, Miguel, etc.).
- Métodos:
 - getStudents(): retorna la lista.
 - postStudent(...): agrega un nuevo estudiante.
 - putStudent(...): reemplaza un estudiante por otro.
 - deleteStudent(...): elimina un estudiante.

Conclusión:

En conclusión, las capas de MVC se comunican de la siguiente manera: el Controlador (StudentController) recibe la petición inicial (desde Main), solicita datos al Modelo



(StudentDatabase y sus objetos Student), y envía esos datos a la Vista (StudentView) para su presentación; cuando la Vista simula una inserción o actualización, devuelve un objeto Student al Controlador, que a su vez modifica el Modelo y vuelve a refrescar la Vista. De este modo, el Controlador actúa como intermediario único, garantizando que la Vista no gestione datos directamente y que el Modelo permanezca independiente de la forma en que se muestran o recogen los datos. Así se mantiene una separación clara de responsabilidades y un flujo unidireccional de información entre las tres capas.